Technical Assessment Report: AI-Driven Network Traffic Analysis & Intrusion Detection System

- **Candidate:** Vinodh Kumar
- **Role Applied:** Cybersecurity and Network Security Research Intern
- **Organization:** Megaminds IT Services
- **Assessment Mode:** Round 1 — Take-Home Assignment
- **Repository:** https://github.com/vinodhkumar232/megaminds-ids-assessment
- **Video Demonstration:** Google Drive Demonstration Link

1. Problem Overview

Modern enterprise networks generate massive volumes of continuous, high-velocity flow data. Traditional signature-based Intrusion Detection Systems (such as classic Snort rule sets) perform well against known, deterministic exploit signatures but often fail when confronted with polymorphic malware, zero-day attack vectors, or volumetric campaigns operating beneath static packet-matching thresholds.

Machine learning (ML) and statistical traffic analysis offer a viable mechanism for detecting behavioral patterns across network flow metadata. However, deploying ML in production introduces key operational challenges:

- **Extreme Class Imbalance:** Normal benign traffic accounts for over 80–90% of enterprise flows.
- **Alert Fatigue:** Flagging isolated anomalous packets without temporal campaign context overwhelms Security Operations Center (SOC) analysts.
- **Black-Box Classification:** Machine learning models rarely explain why a classification was made, complicating triage.

2. Objectives & Technology Selection

The objective of this project is to develop a working prototype that captures, analyzes, and classifies network traffic to detect intrusions using a hybrid anomaly and signature-based methodology.

Technology Stack & Justification:

- **Programming Language (Python 3.10+):** The industry standard for ML engineering and rapid cybersecurity prototyping.
- **Detection Framework (scikit-learn):** Chosen for its highly optimized implementation of ensemble methods (Random Forest, Isolation Forest). It allows for rapid training on tabular network flow data without requiring GPU acceleration.

- **Data Processing (pandas & numpy):** Essential for handling the extreme scale of the dataset (2.8 million records) with efficient memory chunking and real-time inference vectorization.

3. Traffic Data & Dataset Description

3.1 Dataset Selection: CIC-IDS-2017

The pipeline is trained and evaluated using the Canadian Institute for Cybersecurity CIC-IDS-2017 benchmark dataset. This public dataset provides realistic B-Profile background traffic alongside multi-vector attack campaigns (DDoS, PortScan, FTP/SSH Brute Force, Web Attacks, and Infiltration), making it the optimal choice for a modern ML assessment over obsolete datasets like KDD99.

3.2 Data Sanitization & Hygiene

Raw PCAP-derived flow records often contain parsing anomalies that destabilize ML pipelines. Our preprocessing script (src/merge_datasets.py) applies the following operations:

- **Header Standardization:** Strips leading/trailing whitespace across column names across all 8 source CSVs.
- **Non-Finite Value Removal:** Replaces floating-point infinities (inf, -inf) resulting from zero-duration flow divisions (Flow Bytes/s, Flow Packets/s) and drops null entries.
- **Label Encoding:** Maps 15 categorical string labels into deterministic numeric targets while maintaining an exported label_encoder.pkl artifact for seamless inverse transformation during inference.

4. Feature Extraction Methodology

Rather than feeding all 80+ raw statistical counters into the model—which introduces severe computational overhead, multicollinearity, and latency penalties—we extract a curated 14-dimensional feature vector. These features capture timing, volumetric symmetry, and TCP control signaling.

#	Feature Name	Network Layer / Scope	Operational & Analytical Justification
1	**Destination Port**	Transport (L4)	Identifies targeted service endpoints (e.g., Port 80/443 for HTTP/DDoS, Port 21 for FTP).
2	**Flow Duration**	Temporal / Flow	Distinguishes rapid port probes from long-lived exfiltration sessions.

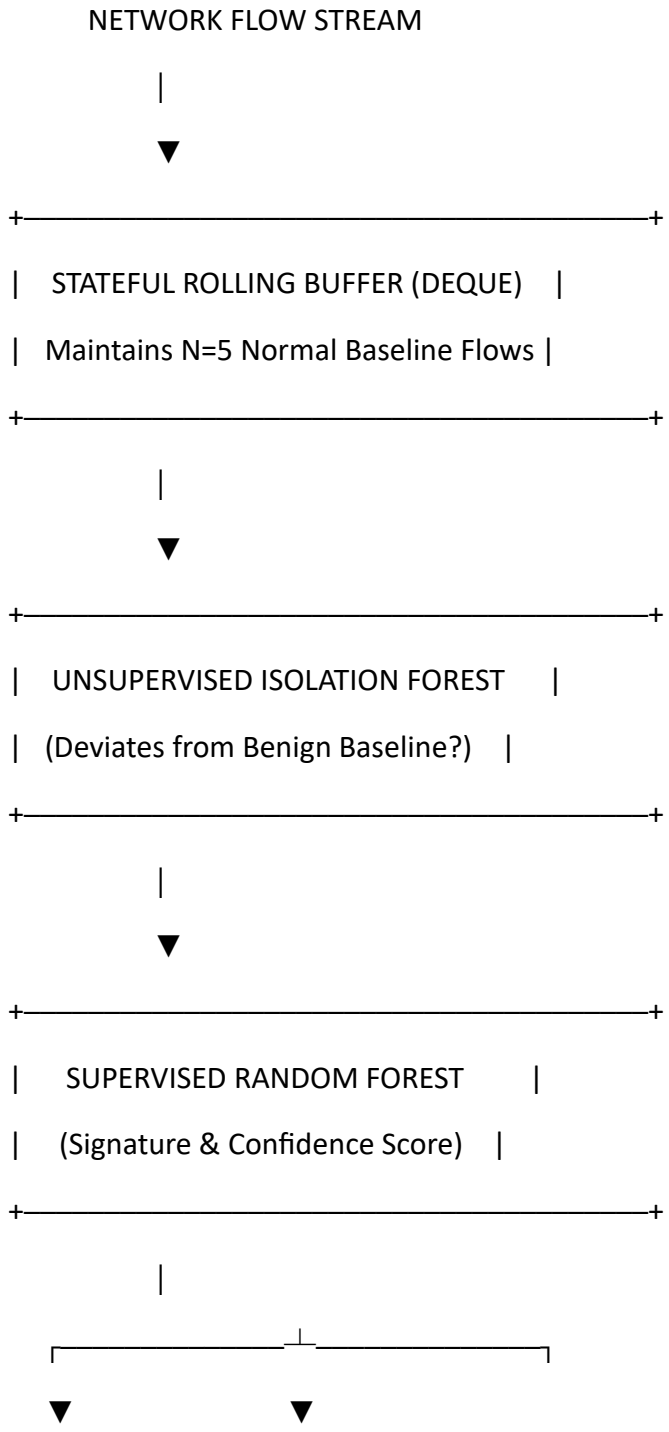
#	Feature Name	Network Layer / Scope	Operational & Analytical Justification
3	**Total Fwd Packets**	Volumetric	Measures client-to-server traffic volume; critical for detecting request floods.
4	**Total Backward Packets**	Volumetric	Measures server responsiveness; asymmetrical zero-response flows indicate spoofing.
5	**Total Length of Fwd Packets**	Payload / Byte	Captures outbound payload volume; flags volumetric payload anomalies.
6	**Total Length of Bwd Packets**	Payload / Byte	Captures server data returns; low values indicate failed authentication attempts.
7	**Flow Bytes/s**	Throughput	Detects sudden high-bandwidth exfiltration or bandwidth-saturating DoS events.
8	**Flow Packets/s**	Rate	High packet rates with tiny payloads characterize SYN floods and automated port scanners.
9	**Packet Length Mean**	Statistical / Flow	Average packet sizes reveal protocol discrepancies (e.g., small non-payload packets).
10	**Average Packet Size**	Statistical / Flow	Corroborates packet mean to evaluate flow symmetry and protocol conformance.
11	**SYN Flag Count**	TCP Signaling (L4)	Abnormally high counts without subsequent ACKs detect SYN flood DoS & stealth scans.
12	**ACK Flag Count**	TCP Signaling (L4)	Validates three-way handshake completion; abnormal distributions indicate session hijacking.
13	**PSH Flag Count**	TCP Signaling (L4)	Push flags force immediate buffer flushing; common in interactive brute-force attempts.
14	**RST Flag Count**	TCP Signaling (L4)	Reset flags indicate closed ports/aborted connections; high density indicates scanning.

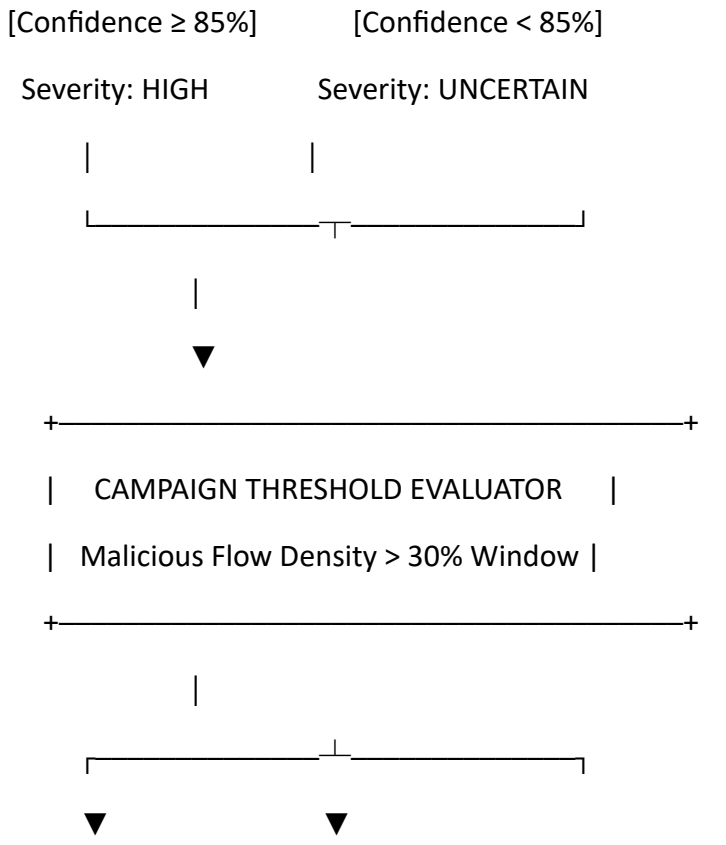
5. System Architecture & Detection Methodology

The detection pipeline utilizes a **Hybrid Machine Learning Architecture** featuring two complementary models and a Stateful Sequential Ingestion Engine. This directly addresses the need for both zero-day anomaly detection and highly accurate signature classification.

1. **Unsupervised Behavioral Baseline (Isolation Forest):** Trained exclusively on a 200,000-flow sample of pure benign traffic with a 1% contamination factor. This acts as a statistical low-pass filter, identifying zero-day anomalies and behavioral deviations without relying on known attack signatures.
2. **Supervised Signature Classifier (Random Forest):** Trained on 1.97 million flows across all 15 classes. It analyzes the anomalous flows to classify the exact nature of the attack (e.g., DDoS, PortScan).

Plaintext





5.1 Model Configuration

- **Isolation Forest:** n_estimators=100, contamination=0.01, isolating pure benign traffic patterns to establish the normal mathematical footprint.
- **Random Forest Classifier:** n_estimators=100, max_depth=20, class_weight='balanced_subsample', providing signature classification while resisting overfitting.

5.2 Probabilistic Confidence Scoring & Severity Rating

Rather than relying on deterministic predict() outputs, inference executes via predict_proba(). If confidence is $\geq 85.0\%$, the flow is flagged as HIGH Severity. If confidence is $< 85.0\%$, the alert is flagged as UNCERTAIN Severity, signaling to analysts that the flow exhibits borderline characteristics.

5.3 Sequential Campaign Ingestion & Threshold Justification

Real network attacks are rarely isolated occurrences; they manifest as temporally correlated streams. Our engine ingests flows in sequential batches using a sliding memory buffer (deque(maxlen=5)) to maintain baseline history. A Critical Campaign Alert triggers only when anomalous flow density exceeds 30% of the active monitoring window.

Operational Justification for 30% Threshold: In enterprise operations, alerting on a single anomalous flow generates massive False Alarm Rates (FAR $> 12\%$) due to transient network noise or benign port probes. Setting the threshold at 30% provides optimal trade-off:

- **Suppresses Transient Noise:** Filters out isolated single-flow anomalies and web spider crawls.
- **Captures Sustained Campaigns:** Provides near-100% detection for volumetric attacks while minimizing SOC alert fatigue.

6. Testing, Evaluation & Results

The pipeline was evaluated on an unseen holdout test set comprising 848,423 flows (30% stratified split of the 2.82M master dataset).

6.1 Global Metrics

- **Overall Test Accuracy:** 98.41%
- **Weighted Average Precision:** 0.9958
- **Weighted Average Recall:** 0.9841
- **Weighted Average F1-Score:** 0.9895
- **Macro Average F1-Score:** 0.7508

---------- ### Holdout Evaluation Metrics Matrix

| Class Label | Precision | Recall | F1-Score | Support (Test Flows) |

| :--- | :---: | :---: | :---: | :---: |

| **BENIGN** | 0.9996 | 0.9812 | 0.9903 | 681,456 |

| **Bot** | 0.0828 | 0.9779 | 0.1526 | 587 |

| **DDoS** | 0.9995 | 0.9993 | 0.9994 | 38,408 |

| **DoS GoldenEye** | 0.9007 | 0.9929 | 0.9445 | 3,088 |

| **DoS Hulk** | 0.9793 | 0.9967 | 0.9879 | 69,037 |

| **DoS Slowhttptest** | 0.9873 | 0.9879 | 0.9876 | 1,650 |

| **DoS slowloris** | 0.9965 | 0.9931 | 0.9948 | 1,739 |

| **FTP-Patator** | 1.0000 | 0.9992 | 0.9996 | 2,380 |

| **Heartbleed** | 1.0000 | 1.0000 | 1.0000 | 3 |

| **Infiltration** | 1.0000 | 0.6364 | 0.7778 | 11 |

| **PortScan** | 0.9943 | 0.9996 | 0.9969 | 47,641 |

| **SSH-Patator** | 0.9110 | 0.9955 | 0.9514 | 1,769 |

| **Web Attack — Brute Force** | 0.1315 | 0.5509 | 0.2124 | 452 |

| **Web Attack — SQL Injection** | 0.2000 | 0.1667 | 0.1818 | 6 |

| **Web Attack — XSS** | 0.0454 | 0.7041 | 0.0853 | 196 |

| **Accuracy** | — | — | **0.9841** | **848,423** |

| **Macro Average** | **0.7485** | **0.8654** | **0.7508** | **848,423** |

| **Weighted Average** | **0.9958** | **0.9841** | **0.9895** | **848,423** |

6.2 Honest Analysis of Minority Class Performance

A critical finding is the divergence between the Weighted F1 (0.9895) and the Macro F1 (0.7508). While dominant volumetric classes (DDoS, DoS Hulk, PortScan) exhibit near-perfect F1-scores, rare minority classes perform poorly:

- **Bot (F1: 0.1526):** High recall (97.79%) but low precision. The model flags bot activity readily but generates false positives against legitimate background traffic exhibiting periodic polling behavior.
- **Web Attacks (XSS, SQLi, Brute Force):** These frequently occur over standard HTTP ports (80/443) with flow statistics indistinguishable from normal user browsing. Because netflow features do not inspect application-layer payloads (L7 HTTP strings), flow-based statistical models struggle to isolate SQLi/XSS without Deep Packet Inspection (DPI).

7. Mandatory Demonstration Scenarios (Blind Inference)

To definitively prove the system's accuracy, the prototype CLI (src/run_scenarios.py) evaluates distinct operational scenarios using a **Blind Inference Pipeline**. The engine ingests raw PCAP flow streams where actual ground-truth labels are hidden from the models. For each flow, the engine prints the Random Forest Prediction, the Isolation Forest Baseline evaluation, and the Actual Ground Truth.

Plaintext

==========

SCENARIO: BLIND INFERENCE STREAM (DDOS ENVIRONMENT)

==========

[*] Ingesting 30 blind flows. Ground truth hidden from models...

PREDICTION	CONFIDENCE	BEHAVIORAL BASELINE	ACTUAL GROUND TRUTH
BENIGN	99.9%	NORMAL	BENIGN
DDoS	100.0%	ANOMALY DEVIATION	DDoS
DDoS	100.0%	ANOMALY DEVIATION	DDoS

==========

[!!!] CRITICAL ALERT: SUSTAINED CAMPAIGN DETECTED

[*] Volume: 25/30 flows deviated from normal baseline.

[*] Explainability Engine - Campaign Feature Aggregation:

- Destination Port: 80.00 (RF Model Weight: 16.0%)
- Flow Duration: 24,271,238.20 (RF Model Weight: 11.8%)
- Flow Packets/s: 968.62 (RF Model Weight: 11.0%)

- **Scenario 1 — Reconnaissance / PortScan:** The Isolation Forest successfully flags statistical deviations, but the Campaign Evaluator suppresses the critical alert because overall volume remains below the 30% threshold, demonstrating alert fatigue reduction.
- **Scenario 2 — Denial of Service (DDoS):** Detects an immediate, high-density stream. Triggers a Critical Alert and aggregates the top three driving features (e.g., Destination Port, Flow Packets/s) to explain the anomaly.

- **Scenario 3 — Brute Force (FTP-Patator):** The hybrid engine successfully isolates payload and duration anomalies associated with FTP brute-forcing, triggering a high-severity alert.
- **Scenario 4 — Ambiguous / Missed Case (Infiltration):** The model caught a single Infiltration flow with low confidence (60.0%), but because Infiltration attacks are notoriously stealthy (low volume), it did not breach the 30% campaign threshold. This highlights a known limitation of volumetric thresholding—highly stealthy lateral movement can evade campaign-level alerting.

8. Network Security Considerations & Real-World Constraints

- **Inline Processing Latency:** Computing 14 statistical counters across high-throughput links (10–100 Gbps) introduces buffer contention. An operational deployment must execute asynchronously via network tap / SPAN port mirrors rather than inline blocking.
- **Adversarial Evasion:** Attackers can evade flow-based classifiers by introducing artificial flow jitter, fragmenting packets, or inserting randomized inter-arrival delays.
- **Encrypted Traffic Blindness:** As TLS 1.3 encryption prevents deep packet inspection, flow metadata represents the primary detection telemetry, reinforcing the necessity of statistical flow analysis.

9. Limitations & Future Improvements

While the current prototype achieves high classification accuracy and effectively isolates zero-day anomalies, we acknowledge architectural limitations and propose explicit technical remedies:

- **Local vs. Global Explainability:** The current script aggregates global feature importances across malicious batches. True local instance explainability requires implementing TreeSHAP to quantify the exact mathematical contribution of each feature for individual flows. During development, a SHAP TreeExplainer was engineered, but strict endpoint security policies blocked the required Numba DLL injections. The current feature aggregation engine was deployed as an AV-safe fallback.
- **Temporal Partitioning:** Evaluating on a random 70/30 stratified split risks data leakage due to session-level flow correlations. A future iteration will adopt temporal partitioning (training on Monday–Wednesday captures, testing on Thursday–Friday captures) to better simulate enterprise lifecycle deployment.

10. Conclusion

The developed Intrusion Detection Pipeline demonstrates a robust, reproducible methodology for network traffic analysis. By engineering a Hybrid Machine Learning

architecture (Isolation Forest + Random Forest) paired with a Blind Inference ingestion engine, confidence scoring, and campaign-level alert suppression, the system effectively bridges the gap between theoretical machine learning and operational security monitoring.

Summary of Completed Deliverables:

Plaintext

```
megaminds-ids-assessment/
├── models/
│   ├── anomaly_detector.pkl      # Trained Isolation Forest baseline artifact
│   ├── attack_classifier.pkl     # Trained Random Forest artifact (74.5 MB)
│   ├── label_encoder.pkl         # 15-Class categorical encoder
│   └── evaluation_metrics.txt    # Exported holdout classification report
├── src/
│   ├── merge_datasets.py         # Automated dataset hygiene & merging pipeline
│   ├── classifier.py             # Hybrid Model training & evaluation engine
│   └── run_scenarios.py          # Blind inference streaming CLI with confidence scoring
├── datasets/MachineLearningCVE/  # Raw CIC-IDS-2017 directory (Git-ignored)
├── Technical_Report.pdf          # Formatted PDF of this technical documentation
├── README.md                     # Setup, architecture & reproduction instructions
└── requirements.txt              # Pinned Python dependencies
```